

◦ CSE 1113

Computer Programming

Sumaya Kazary

Assistant Professor

Department of Computer Science and Engineering

Dhaka University of Engineering & Technology, Gazipur

Acknowledgement

Thanks to the authors of all the books and online tutorials used in this slide.



Lecture -2 (SK)

Operators and Expressions

Expressions and Statements

- Expression

- Combination of constants, variables, operators, and function calls

- Example `a+b`

`3.0*x - 9.66553`

`tan(angle)`

- Statement

- An expression terminated with a semicolon

- Example

`sum = x + y + z;`

`printf("Welcome to DUET!");`

Operators in C

- C supports a rich set of operators.
- Operators are used in programs to manipulate data and variables.
- They usually form a part of the mathematical or logical expressions.
- C operators are classified into a number of categories. They include:
 1. Arithmetic operators
 2. Relational operators
 3. Logical operators
 4. Assignment operators
 5. Increment and Decrement operators
 6. Conditional operators
 7. Bitwise operators
 8. Special operators

Arithmetic Operators

- Addition + `sum = num1 + num2;`
- Subtraction - `age = 2007 - my_birth_year;`
- Multiplication * `area = side1 * side2;`
- Division / `avg = total / number;`
- Modulus % `lastdigit = num % 10;`

- Modulus returns remainder of division between two *integers*

- Example `5%2` returns a value of 1

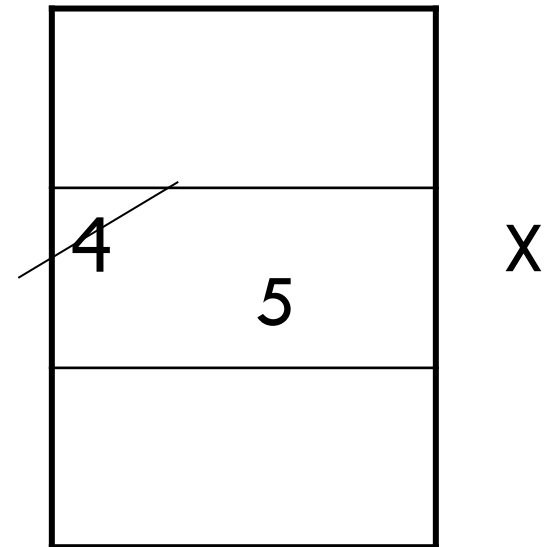
- Binary vs. Unary operators

- All the above operators are binary (why)

- - is an unary operator, e.g., `a = -3 * -4`

Arithmetic Operators (cont'd)

- Note that ' $\text{id} = \text{exp}$ ' means assign the result of exp to id , so
- $X = X + 1$ means
 - first perform $X + 1$ and
 - Assign the result to X
- Suppose X is 4, and
- We execute $X = X + 1$



Integer division vs Real division

- Division between two integers results in an integer.
- The result is truncated, not rounded

- Example:

`int A=5/3;` → A will have the value of 1

`int B=3/6;` → B will have the value of 0

- To have floating point values:
 - If one or two operands are floating-point type : the result is floating-point type

`double A=5.0/3;` → A will have the value of 1.666

`double B=3.0/6.0;` → B will have the value of 0.5

Precedence of Arithmetic Operators

Mixed operations:

`int a=4+6/3*2; → a=?`

$$a = 4 + 2 * 2 = 4 + 4 = 8$$

`int b=(4+6)/3*2; → b=?`

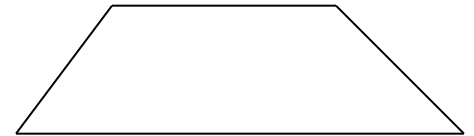
$$b = 10 / 3 * 2 = 3 * 2 = 6$$

Precedence	Operator	Associativity
1	Parentheses: ()	Innermost first
2	Unary operators: + - (type)	Right to left
3	Binary operators: * / %	Left to right
4	Binary operators: + -	Left to right
5	assign =	Right to left

Exercise

- Compute the following
 - $2*(3+2)$
 - $2*3+2$
 - $6-3*2$
- Area of trapezoid

$$area = \frac{base * (height_1 + height_2)}{2}$$



`area = 1.0/2*base*(height_1+height_2);`

Exercise

- Write a C statement to compute the following

$$f = \frac{x^3 - 2x^2 + x - 6.3}{x^2 + 0.05x + 3.14}$$

`f = (x*x*x-2*x*x+x-6.3)/(x*x+0.05*x+3.14);`

- Write a C statement to compute the following

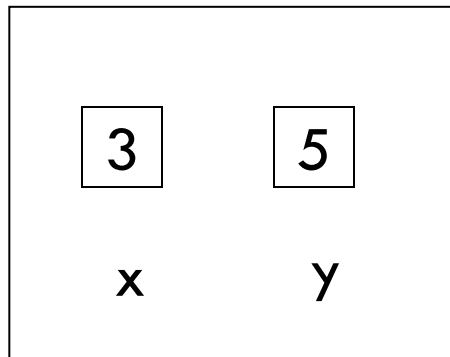
$$Tension = \frac{2m_1m_2}{m_1 + m_2} \times g$$

`Tension = 2*m1*m2 / m1 + m2 * g;` **WRONG**

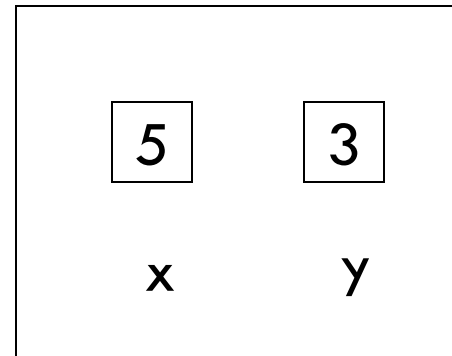
`Tension = 2*m1*m2 / (m1 + m2) * g`

Exercise: swap

- Write a set of statements that swaps the contents of variables x and y



Before



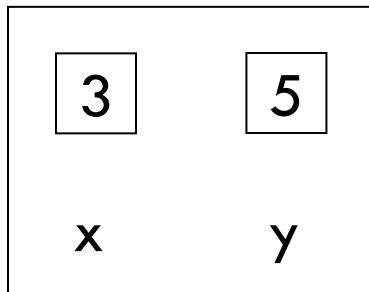
After

Exercise: swap

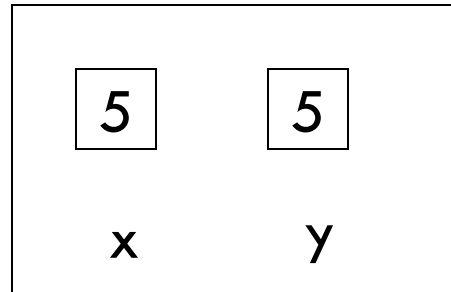
First Attempt

$x=y;$

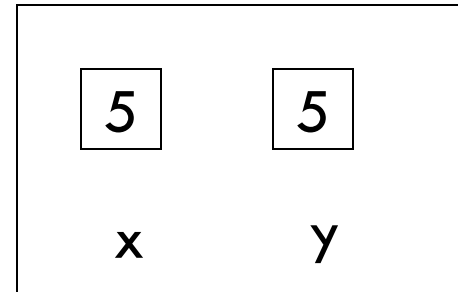
$y=x;$



Before



After $x=y$



After $y=x$

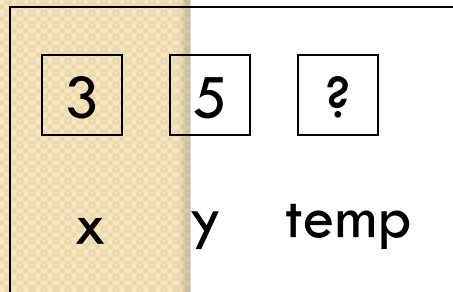
Exercise: swap

Solution

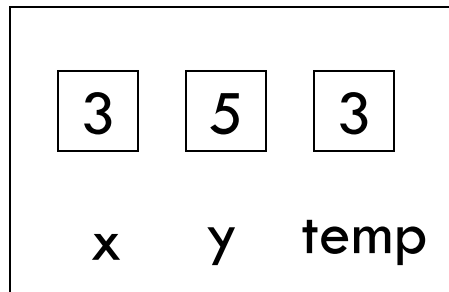
```
temp= x;
```

```
x=y;
```

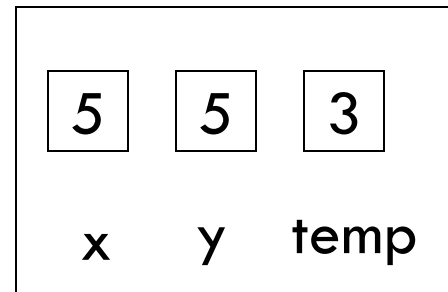
```
y=temp;
```



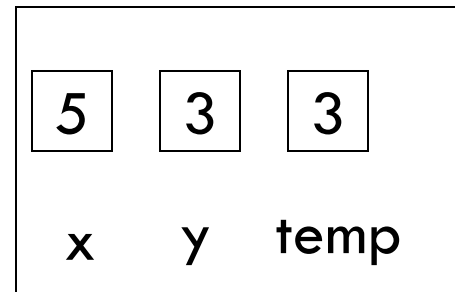
Before



after temp=x



after x=y



after y = temp

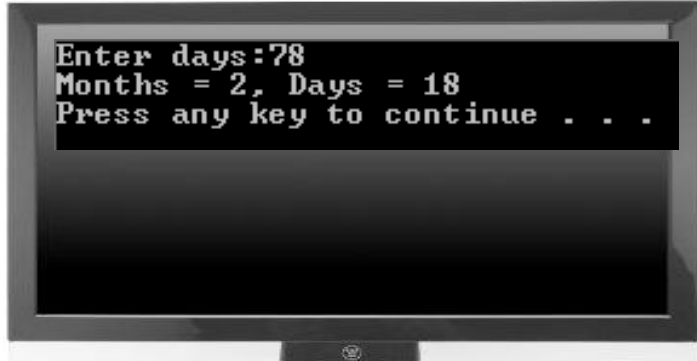
Arithmetic operators: Example

- The program converts a given number of days into months and days

```
#include <stdio.h>
void main()
{
    int months, days;

    printf("Enter days:");
    scanf("%d", &days);

    months = days / 30;
    days = days % 30;
    printf("Months = %d, Days = %d", months, days);
}
```



Enter days:78
Months = 2, Days = 18
Press any key to continue . . .

Exercise: Arithmetic operations

- Show the memory snapshot after the following operations by hand

```
int a, b, c=5;
double x, y;
a = c * 2.5;
b = a % c * 2 - 1;
x = (5 + c) * 2.5;
y = x - (-3 * a) / 2;
```

Write a C program and print out the values of a, b, c, x, y and compare them with the ones that you determined by hand.

?	a
?	b
5	c
?	x
?	y

a = 12 b = 3 c = 5 x = 25.0000 y = 43.0000

Exercise: Arithmetic operations

- Show how C will perform the following statements and what will be the final output?

```
int a = 6, b = -3, c = 2;  
c = a - b * (a + c * 2) + a / 2 * b;  
printf("Value of c = %d \n", c);
```


Step-by-step show how C will perform the operations

$c = 6 - -3 * (6 + 2 * 2) + 6 / 2 * -3;$

$c = 6 - -3 * (6 + 4) + 3 * -3$

$c = 6 - -3 * 10 + -9$

$c = 6 - -30 + -9$

$c = 36 + -9$

$c = 27$

- **output:**

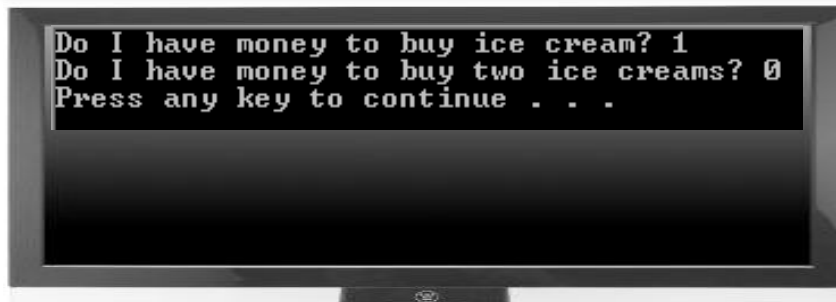
Value of $c = 27$

Relational operators

- We often compare two quantities and depending on their relation, take certain decisions.
- For example you have 1000 Tk. in your pocket, then when the question is:
 - Do I have enough money to buy an ice cream
 - In C this question can be written as follows

```
#include <stdio.h>
void main()
{
    int icecream_price = 950;
    int pocket_money = 1000;

    printf("Do I have money to buy ice cream? %d \n", icecream_price < pocket_money);
    printf("Do I have money to buy two ice creams? %d \n", 2*icecream_price < pocket_money);
}
```



YES (TRUE) any non-zero value

NO (FALSE)

Relational operators

- Relational operators are used in decision statements such as **if** and **while** to decide the course of action of a running program.

Operator	Meaning
<	Is less than
<=	Is less than or equal to
>	Is greater than
>=	Is greater than or equal to
==	Is equal to
!=	Is not equal to

- A simple relational expression contains only one relational operator:

arithm_expr1 **rel_op** *arithm_expr2*

4.5 <= 10 TRUE

4.5 < -10 FALSE

-35 >= 0 FALSE

10 < 7+5 TRUE

a + b == c + d TRUE // if a + b is equal to c + d

Logical operators

- In addition to the relational operators, C has the following three *logical operators*.

&&	Meaning logical	AND
	Meaning logical	OR
!	Meaning logical	NOT

- They are used when we need to test more than one conditions to make decisions.
 - An example is:
- `a > b && x == 10`
- The compound expression is true when `a > b` and `x == 10` is true.

```
if(age > 55 && salary < 1000){...}  
if(number < 0 && number > 100){...}
```

Logical operators

op-1	op-2	Value of the expression	
		op-1 && op-2	op-1 op-2
Non-zero	Non-zero	1	1
Non-zero	0	0	1
0	Non-zero	0	1
0	0	0	0

- *Relative precedence* of the relational and logical operators is as follows

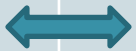
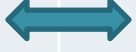
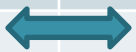
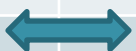
Highest priority	!
	>, >=, <, <=
	==, !=
	&&
Lowest priority	

Assignment operators

- We have already applied the usual assignment operator.
- In addition, C has a set of shorthand assignment operators of the form

v *op* = *expr*;

is equivalent to **$v = v$ *op* (*expr*)**

Statement with simple assignment operator		Statement with shorthand operator
$a = a + 1$		$a += 1$
$a = a - 1$		$a -= 1$
$a = a * (n+1)$		$a *= (n+1)$
$a = a / (n+1)$		$a /= (n+1)$
$a = a \% b$		$a \% = b$

Use of shorthand operator *=

- The program prints a sequence of squares of numbers starting from 2.
- The statement

`a *= a;`



`a = a * a;`

Replaces the current value of **a** by its square. When the value of **a** becomes equal or greater than **N** (=100) the **while** is terminated.

```
#include <stdio.h>
#define N 100

void main(){
    int a;
    a = 2;

    while(a < N){
        printf("%d\n", a);
        a *= a;
    }
}
```



Increment and decrement operators

- C allows two very useful operators not generally found in other languages.
- There are the increment and decrement operators:
++ and **--**
- The operator **++** adds 1 to the operand,
 - **--** subtracts 1.

++m; is equivalent to m = m + 1; (or m+= 1)

--m; is equivalent to m = m - 1; (or m-= 1)

Prefix operator

m = 5;

y = ++m;

m is 6 and y is 5

Postfix operator

m = 5;

y = m++;

m is 6 and y is 6

- **Example: m = n++ - j + 10; Old value of n is used in evaluation.**

Conditional Operator

- A ternary operator pair “**?** **:**” is available in C to construct conditional expressions of the form:

exp1 ? exp2 : exp3

where ***exp1***, ***exp2***, and ***exp3*** are expressions.

- ***exp1*** is a condition and is evaluated first.
 - If it is true, then the expression ***exp2*** is evaluated and becomes the value of expression.
 - Otherwise ***exp3*** becomes the value of expression.

a = 10;

b = 15;

x = (a > b) ? a : b;

Bitwise operator

- Bitwise operators are used for testing the bits, or shifting them right or left.
- Bitwise operators are not applied to **float** or **double**.

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
<<	Shift left
>>	Shift right



inputs	&
0 & 0	0
0 & 1	0
1 & 0	0
1 & 1	1

inputs	
0 & 0	0
0 & 1	1
1 & 0	1
1 & 1	1

inputs	^
0 & 0	0
0 & 1	1
1 & 0	1
1 & 1	0

Special operators

- C supports some special operators such as
 - comma operator ,
 - `value = (x = 10, y = 5, x + y);`
 - **while** `(c = getchar(), c != '10')`
 - `t = x, x = y, y = t;`
 - **sizeof** operator
 - `m = sizeof(sum);`
 - `n = (long int);`
 - `K = sizeof(235L)`
 - pointer operators (**&** and *****) and
 - member selection operators (**.** and **->**)

Summary example

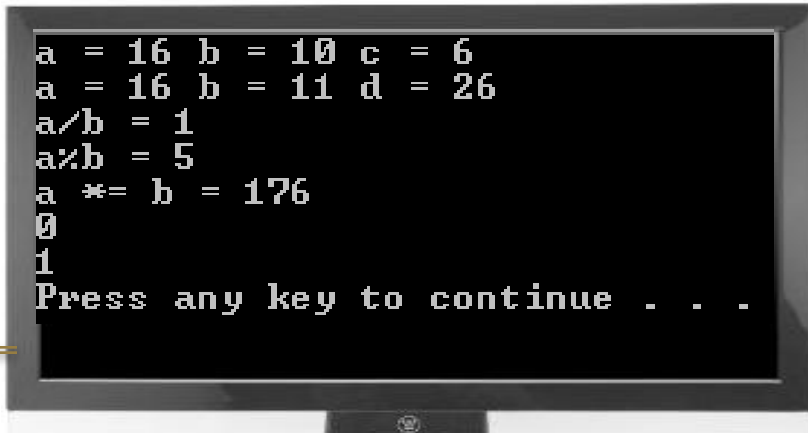
```
#include <stdio.h>

void main(){
    int a, b, c, d;
    a = 15;
    b = 10;
    c = ++a - b;

    printf("a = %d b = %d c = %d\n", a, b, c);

    d = b++ + a;

    printf("a = %d b = %d d = %d\n", a, b, d);
    printf("a/b = %d\n", a/b);
    printf("a%%b = %d\n", a%b);
    printf("a *= b = %d\n", a *= b);
    printf("%d\n", (c > d) ? 1 : 0);
    printf("%d\n", (c < d) ? 1 : 0);
}
```



```
a = 16 b = 10 c = 6
a = 16 b = 11 d = 26
a/b = 1
a%b = 5
a *= b = 176
0
1
Press any key to continue . . .
```

Arithmetic expressions

- An arithmetic expression is a combination of variables, constants and operators arranged as per syntax of the language.
- **C does not have an operator for exponentiation.** $y = x * x$

Algebraic expression	C expression
$a \cdot b - c$	$a * b - c$
$(m+n) \cdot (x+y)$	$(m+n) * (x+y)$
	$A * b / c$
	$3 * x * x + 2 * x + 1$
	$x / y + c$

Precedence of Arithmetic Operators

- An arithmetic expression without parentheses will be evaluated from left to right using the rules of precedence of operators. There are two priority levels:
 - **High priority** * / %
 - **Low priority** + -
- The basic evaluation procedure includes 'two' left-to-right passes.

$$x = a - b/3 + 2*c - 1;$$

➤ when **a = 9**, **b = 12**, and **c = 3**,

$$x = 9 - 12/3 + 3*2 - 1$$

- First pass

1. $x = 9 - 4 + 3 * 2 - 1$

2. $x = 9 - 4 + 6 - 1$

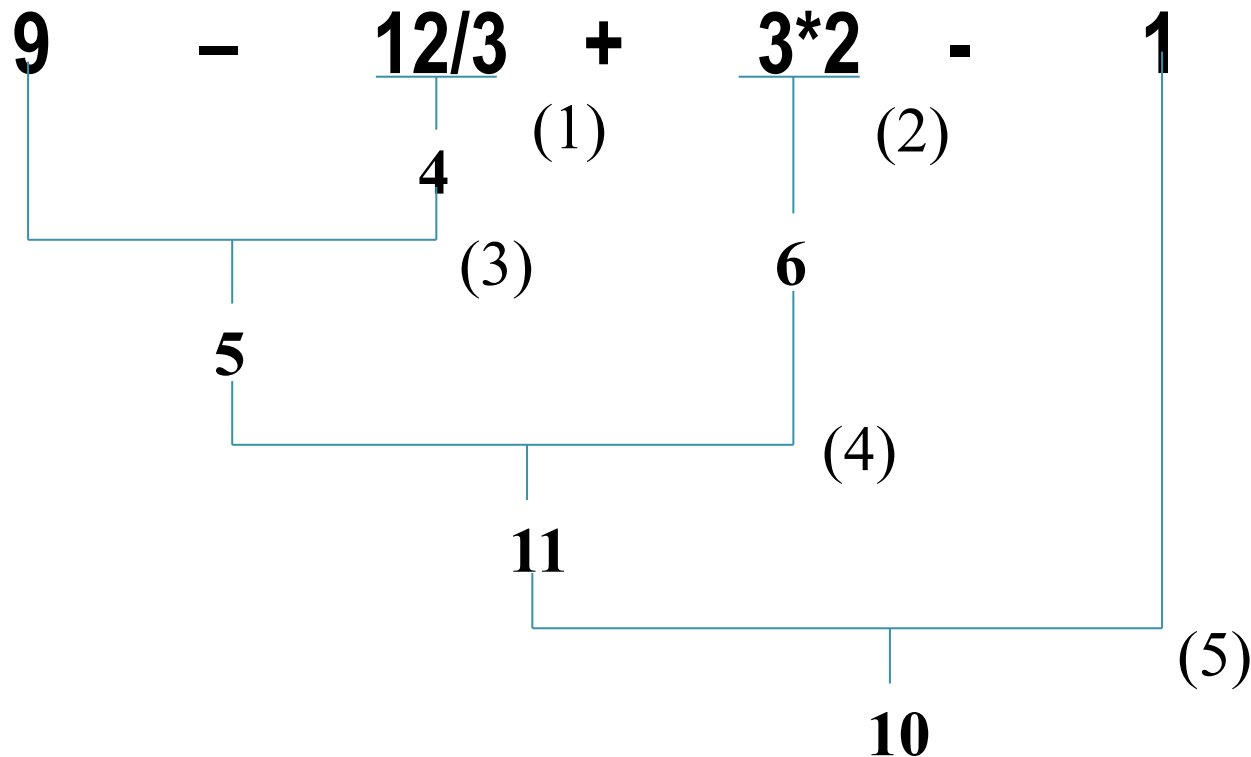
- Second pass

1. $x = 5 + 6 - 1$

2. $x = 11 - 1$

3. $x = 10$

Precedence of arithmetic operators



Plot similar diagram for $9 - 12/(3 + 3)*(2-1)$.
Consider that () has the highest priority!

Some computational problems

- **Computers give approximate values for real numbers** and the error due to such approximations may lead to serious problems:

`a = 1.0/3.0;`

`b = a * 3.0;`

- We know that $(1.0/3.0) * 3.0$ is equal to 1. But there is not guarantee that the value of **b** computed in a program will equal 1.
- **Division by zero.** Any attempt to divide a number by zero will result in abnormal termination of the program. (We should avoid it)
- **Overflow and underflow errors.** It is our responsibility to guarantee that operands are of the correct type and range.

Some computational problems

- Output of the program shows round-off errors that can occur in computation of floating point numbers.
- We know that the sum of n terms of $1/n$ is 1 . However, due to errors in floating point representation, the result is not always 1 .

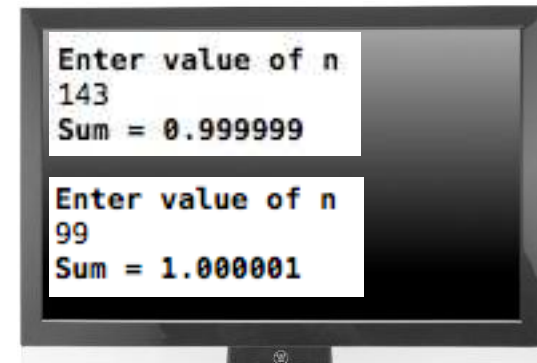
```
#include <stdio.h>

void main(){
    float sum, n, term;
    int count = 1;

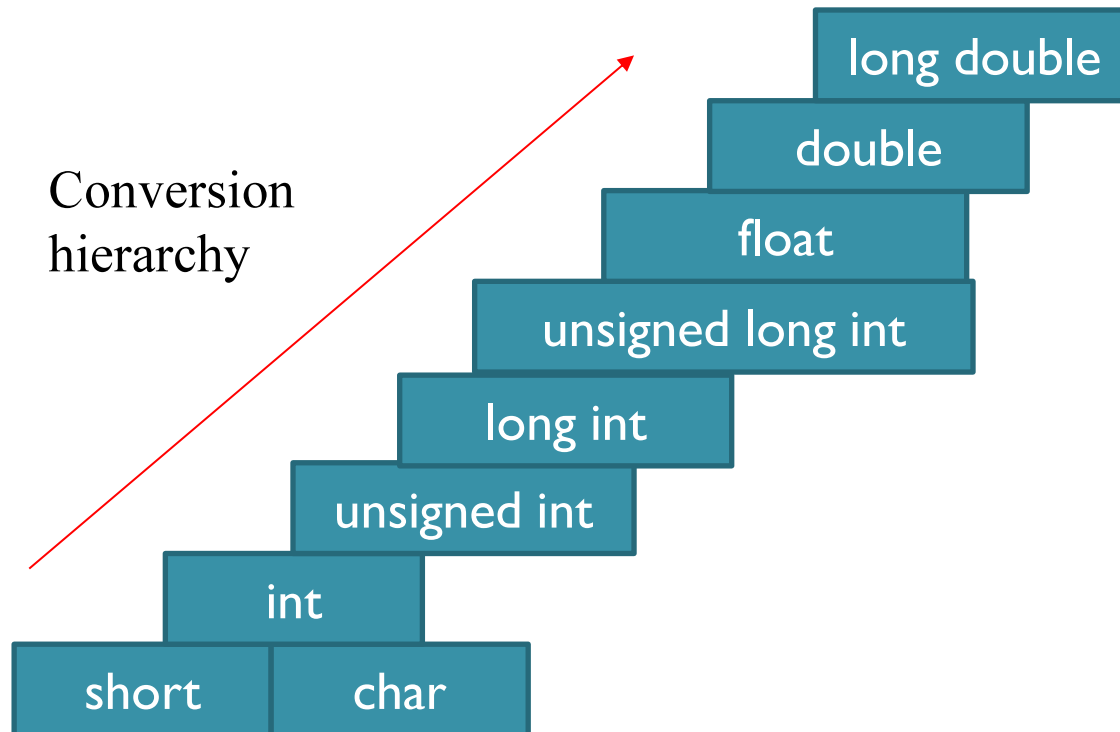
    sum = 0;
    printf("Enter value of n\n");
    scanf("%f", &n);

    term = 1.0/n;

    while(count <= n){
        sum = sum + term;
        count++;
    }
    printf("Sum = %f\n", sum);
}
```



Type conversions in expressions



Type conversions in expressions

- Sometimes we want to force a type conversion in a way that is different from the automatic conversion.

- Example:

```
float ratio;  
int females_number = 7;  
int males_number = 5;  
ratio = females_number / males_number;  
           // the ratio will be 1  
ratio = (float ) females_number / males_number;  
           // the ratio is 1.4
```

The operator (`float`) converts the `females_number` to floating point for the purpose of evaluation of the expression.

The general form of a cast (explicit conversion) is
`(type-name)expression`

Type conversions in expressions

- Casting can be used to round-off a given value. Consider the following statement:

```
x = (int) (y + 0.5);
```

If y is 27.6, $y+0.5$ is 28.1 and on casting, the result becomes 28, the value that is assigned to x .

Example	Action
<code>x = (int) 7.5</code>	7.5 is converted to integer by truncation.
<code>a = (int) 21.3 / (int) 4.5</code>	Evaluated as 21/4 and result would be 5.
<code>b = (double) sum/n</code>	Division is done in floating mode.
<code>y = (int) (a + b)</code>	The result of $a + b$ is converted to integer
<code>z = (int) a + b</code>	a is converted to integer and the added to b
<code>p = cos((double) x)</code>	Converts x to double before using it.

Example

- A program shows a cast to evaluate the equation

$$sum = \sum_{i=1}^n (1/i)$$

```
#include <stdio.h>

int main(){
    float sum;
    int n;

    sum = 0;

    for (n = 1; n <= 10; ++n) {
        sum = sum + 1/(float)n;
        printf("%2d %6.4f\n", n, sum);
    }
}
```



1	1.0000
2	1.5000
3	1.8333
4	2.0833
5	2.2833
6	2.4500
7	2.5929
8	2.7179
9	2.8290
10	2.9290

Operator precedence and associativity

- Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others;

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type) * & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %>> = <<= &= ^= =	Right to left
Comma	,	Left to right

Mathematical functions `#include <math>`

Trigonometric

`double acos(double x)`

Compute arc cosine of x .

`double asin(double x)`

Compute arc sine of x .

`double atan(double x)`

Compute arc tangent of x .

`double atan2(double y, double x)`
Compute arc tangent of y/x , using the signs of both arguments to determine the quadrant of the return value.

`double cos(double x)`

Compute cosine of angle in radians.

`double sin(double x)`

Compute sine of angle in radians.

`double tan(double x)`

Compute tangent of angle in radians.

Hyperbolic functions

`double sinh(double x)`

Compute the hyperbolic sine of x .

`double cosh(double x)`

Compute the hyperbolic cosine of x .

`double tanh(double x)`

Compute the hyperbolic tangent of x .

Other functions

`double sqrt(double x)`

Compute the square root of x .

`double exp(double x)`

Compute exponential of x

`double fabs (double x )`

Compute absolute value of x .

`double floor(double x)`

Get largest integral value less than x .

`double ceil(double x)`

Get smallest integral value that exceeds x .

`double log(double x)`

Natural log of x , $x > 0$

`double pow (double x, double y)`

Compute x raised to the power y .

`double frexp(double x, int *exp_ptr)`

Breaks down x into mantissa and exponent of no.

`void srand(unsigned seed)`

Set a new seed for the random number generator (rand).

`labs(long n)`

Find absolute value of long integer n .

Salesman's salary

- A computer manufacturing company has the following monthly compensation policy to their sales-persons:
 - Minimum base salary : 1500.00
 - Bonus for every computer sold : 200.00
 - Commission on the total monthly sales : 2 per cent

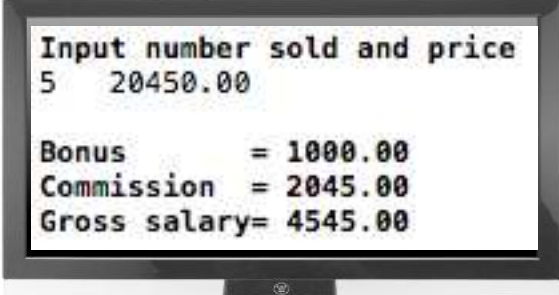
```
#include <stdio.h>
#define BASE_SALARY 1500.00
#define BONUS_RATE 200.00
#define COMMISSION 0.02
int main(){
    int quantity;
    float gross_salary, price;
    float bonus, commission;

    printf("Input number sold and price\n");
    scanf("%d %f", &quantity, &price);

    bonus    = BONUS_RATE * quantity;
    commission = COMMISSION * quantity * price;
    gross_salary = BASE_SALARY + bonus + commission;

    printf("\n");
    printf("Bonus      = %6.2f\n", bonus);
    printf("Commission = %6.2f\n", commission);
    printf("Gross salary= %6.2f\n", gross_salary);
    return 0;
}
```

- The price of computers are changing, the sales price of each computer is fixed at the beginning of every month



```
Input number sold and price
5  20450.00

Bonus      = 1000.00
Commission = 2045.00
Gross salary= 4545.00
```




Thank You